

RV32I

IFU

- 取值

PC/Redirect
Control

- 1.当前PC
- 2.next_pc选择
- 3.Redirect优先级

Fetch Request
Engine

- 1.发起读请求
- 2.等待总线握手
- 3.持有
outstanding
request

Response Buffer

- 1.锁存返回/pc/
错误
- 2.支持
kill/discard

Fetch Check
Logic

- 1.PC对齐检查
- 2.Access fault整
理
- 3.可选PMP检查

IFU control FSM: 协调总控

状态转移

Text

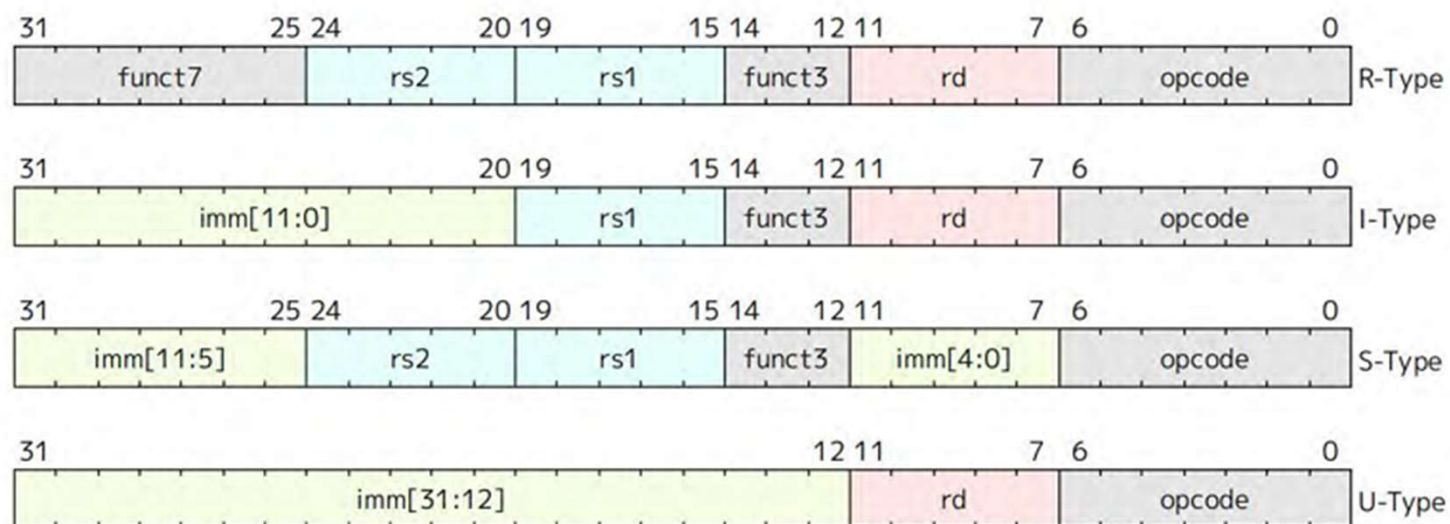
```
RESET
↓
IDLE
  ↓ (start fetch)
SEND_REQ
  ↓ (handshake done)
WAIT_RSP
  ├── rsp_err      -> RESP_ERR
  ├── rsp_ok       -> RESP_OK
  └── redirect_kill -> WAIT_RSP (mark killed)
RESP_OK
↓
HOLD_VALID
  ↓ (decode accept)
IDLE
RESP_ERR
↓
EXCEPTION_WAIT
↓
IDLE
```

典型优先级：

1. reset
2. debug entry/exit
3. exception
4. `mret`
5. branch/jump redirect
6. normal `pc+4`

IDU

- 译码



R-type

- [31:25] **funct7** | [24:20] **rs2** | [19:15] **rs1** | [14:12] **funct3** | [11:7] **rd** | [6:0] **opcode**

指令	类型	opcode	funct3	funct7	功能伪代码
ADD	R	0110011	000	0000000	$R[rd] = R[rs1] + R[rs2]$
SUB	R	0110011	000	0100000	$R[rd] = R[rs1] - R[rs2]$
SLL	R	0110011	001	0000000	$R[rd] = R[rs1] \ll R[rs2][4:0]$
SLT	R	0110011	010	0000000	$R[rd] = (\text{signed}(R[rs1]) < \text{signed}(R[rs2])) ? 1 : 0$
SLTU	R	0110011	011	0000000	$R[rd] = (\text{unsigned}(R[rs1]) < \text{unsigned}(R[rs2])) ? 1 : 0$
XOR	R	0110011	100	0000000	$R[rd] = R[rs1] \wedge R[rs2]$
SRL	R	0110011	101	0000000	$R[rd] = R[rs1] \gg R[rs2][4:0]$
SRA	R	0110011	101	0100000	$R[rd] = \text{signed}(R[rs1]) \ggg R[rs2][4:0]$
OR	R	0110011	110	0000000	$R[rd] = R[rs1] \mid R[rs2]$
AND	R	0110011	111	0000000	$R[rd] = R[rs1] \& R[rs2]$

I-type: 算术立即数指令

- [31:20] **imm**[11:0] | [19:15] **rs1** | [14:12] **funct3** | [11:7] **rd** | [6:0] **opcode**
- $\text{imm_i} = \text{SignExt}(\text{inst}[31:20])$
- $\text{shamt} = \text{inst}[24:20]$

指令	类型	opcode	funct3	funct7高位	功能伪代码
ADDI	I	0010011	000	-	$R[\text{rd}] = R[\text{rs1}] + \text{imm_i}$
SLTI	I	0010011	010	-	$R[\text{rd}] = (\text{signed}(R[\text{rs1}]) < \text{signed}(\text{imm_i})) ? 1 : 0$
SLTIU	I	0010011	011	-	$R[\text{rd}] = (\text{unsigned}(R[\text{rs1}]) < \text{unsigned}(\text{imm_i})) ? 1 : 0$
XORI	I	0010011	100	-	$R[\text{rd}] = R[\text{rs1}] \wedge \text{imm_i}$
ORI	I	0010011	110	-	$R[\text{rd}] = R[\text{rs1}] \vee \text{imm_i}$
ANDI	I	0010011	111	-	$R[\text{rd}] = R[\text{rs1}] \& \text{imm_i}$
SLLI	I	0010011	001	0000000	$R[\text{rd}] = R[\text{rs1}] \ll \text{shamt}$
SRLI	I	0010011	101	0000000	$R[\text{rd}] = R[\text{rs1}] \gg \text{shamt}$
SRAI	I	0010011	101	0100000	$R[\text{rd}] = \text{signed}(R[\text{rs1}]) \ggg \text{shamt}$

LOAD指令

- opcode = 0000011
- imm_i = SignExt(inst[31:20])
- addr = R[rs1] + imm_i

指令	类型	funct3	数据类型	功能伪代码
LB	I	000	byte signed	$R[rd] = \text{SignExt}(\text{MEM}[\text{addr}][7:0])$
LH	I	001	half signed	$R[rd] = \text{SignExt}(\text{MEM}[\text{addr}][15:0])$
LW	I	010	word	$R[rd] = \text{MEM}[\text{addr}][31:0]$
LBU	I	100	byte unsigned	$R[rd] = \text{ZeroExt}(\text{MEM}[\text{addr}][7:0])$
LHU	I	101	half unsigned	$R[rd] = \text{ZeroExt}(\text{MEM}[\text{addr}][15:0])$

Store指令

- opcode = 0100011
- imm_s = SignExt({inst[31:25], inst[11:7]})
- addr = R[rs1] + imm_s

指令	类型	funct3	数据类型	功能伪代码
SB	S	000	byte	MEM[addr][7:0] = R[rs2][7:0]
SH	S	001	half	MEM[addr][15:0] = R[rs2][15:0]
SW	S	010	word	MEM[addr][31:0] = R[rs2][31:0]

Branch指令

- opcode = 1100011
- imm_b = SignExt({inst[31], inst[7], inst[30:25], inst[11:8], 1'b0})
- target = PC + imm_b

指令	类型	funct3	比较类型	功能伪代码
BEQ	B	000	相等	if (R[rs1] == R[rs2]) PC = target else PC = PC + 4
BNE	B	001	不等	if (R[rs1] != R[rs2]) PC = target else PC = PC + 4
BLT	B	100	有符号	if (signed(R[rs1]) < signed(R[rs2])) PC = target else PC = PC + 4
BGE	B	101	有符号	if (signed(R[rs1]) >= signed(R[rs2])) PC = target else PC = PC + 4
BLTU	B	110	无符号	if (unsigned(R[rs1]) < unsigned(R[rs2])) PC = target else PC = PC + 4
BGEU	B	111	无符号	if (unsigned(R[rs1]) >= unsigned(R[rs2])) PC = target else PC = PC + 4

U-type

LUI

Text

```
opcode = 0110111
imm_u = {inst[31:12], 12'b0}
R[rd] = imm_u
```

AUIPC

Text

```
opcode = 0010111
imm_u = {inst[31:12], 12'b0}
R[rd] = PC + imm_u
```

指令	类型	opcode	功能伪代码
LUI	U	0110111	$R[rd] = \text{imm_u}$
AUIPC	U	0010111	$R[rd] = PC + \text{imm_u}$

Jump指令

JAL

编码格式:

Text

opcode = 1101111
imm_j = SignExt({inst[31], inst[19:12], inst[20], inst[30:21], 1'b0})

伪代码:

Text

tmp = PC + 4
PC = PC + imm_j
R[rd] = tmp

JALR

编码格式:

Text

opcode = 1100111
funct3 = 000
imm_i = SignExt(inst[31:20])

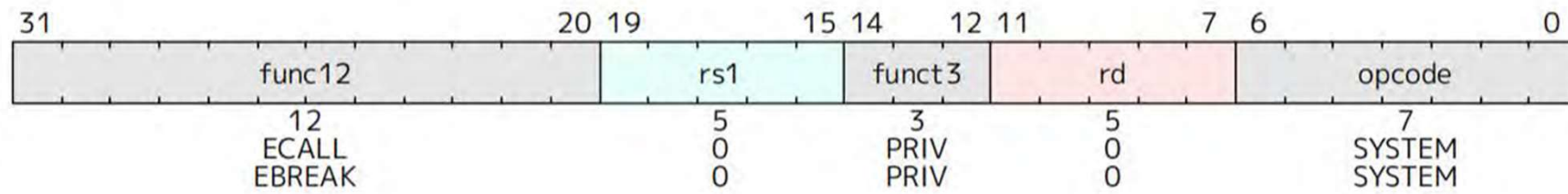
伪代码:

Text

tmp = PC + 4
PC = (R[rs1] + imm_i) & 0xFFFF_FFFE
R[rd] = tmp

指令	类型	opcode	funct3	功能伪代码
JAL	J	1101111	-	R[rd] = PC + 4; PC = PC + imm_j
JALR	I	1100111	000	R[rd] = PC + 4; PC = (R[rs1] + imm_i) & ~1

sys



指令约束

指令	约束	目的
LUI	rd=x0	指定用于将来的标准使用
AUIPC	rd=x0	
ADDI	rd=x0, and either rs1≠x0 or imm≠0	
ANDI	rd=x0	
ORI	rd=x0	

FENCE	rd=rs1=x0, fm=0, pred=W, succ=0	终止
SLTI	rd=x0	指定自定义使用
SLTIU	rd=x0	
SLLI	rd=x0	
SRLI	rd=x0	
SRAI	rd=x0	
SLT	rd=x0	
SLTU	rd=x0	

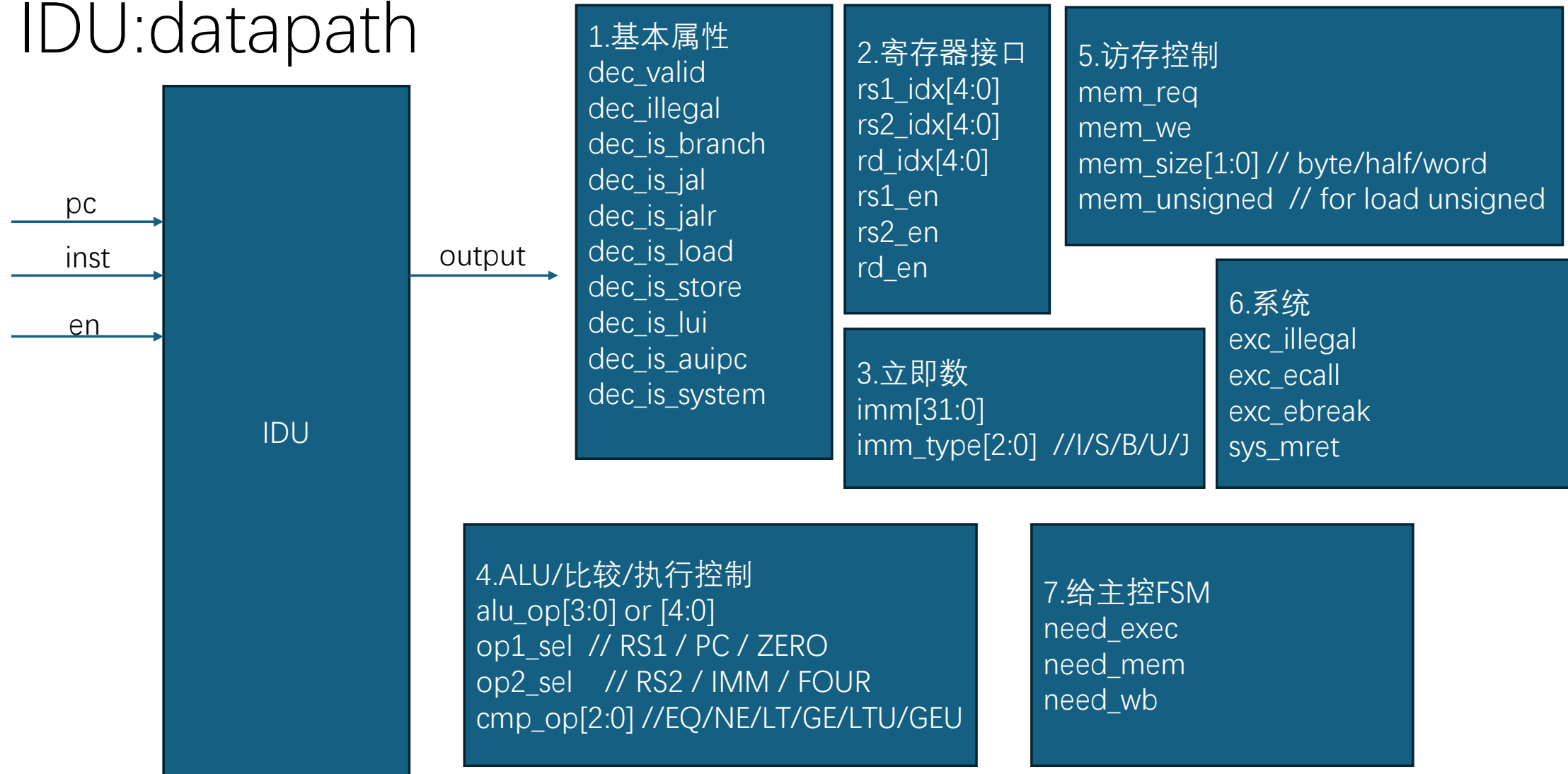
XORI	rd=x0	
ADD	rd=x0, rs1≠x0	
ADD	rd=x0, rs1=x0, rs2≠x2-x5	
ADD	rd=x0, rs1=x0, rs2=x2-x5	(rs2=x2) NTL. P1 (rs2=x3) NTL. PALL (rs2=x4) NTL. S1 (rs2=x5) NTL. ALL
SUB	rd=x0	指定用于将来的标准使用
AND	rd=x0	
OR	rd=x0	
XOR	rd=x0	
SLL	rd=x0	
SRL	rd=x0	
SRA	rd=x0	
FENCE	rd=x0, rs1≠x0, fm=0, and either pred=0 or succ=0	
FENCE	rd≠x0, rs1=x0, fm=0, and either pred=0 or succ=0	
FENCE	rd=rs1=x0, fm=0, pred=0, succ≠0	
FENCE	rd=rs1=x0, fm=0, pred≠W, succ=0	

数据处理一览表

```
opcode    = inst[6:0]
rd         = inst[11:7]
funct3     = inst[14:12]
rs1        = inst[19:15]
rs2        = inst[24:20]
funct7     = inst[31:25]

imm_I      = SignExt(inst[31:20])
imm_S      = SignExt({inst[31:25], inst[11:7]})
imm_B      = SignExt({inst[31], inst[7], inst[30:25], inst[11:8], 1'b0})
imm_U      = {inst[31:12], 12'b0}
imm_J      = SignExt({inst[31], inst[19:12], inst[20], inst[30:21], 1'b0})
shamt      = inst[24:20]
```

IDU:datapath



立即数处理

- opcode = instr[6:0]
- rd = instr[11:7]
- funct3 = instr[14:12]
- rs1 = instr[19:15]
- rs2 = instr[24:20]
- funct7 = instr[31:25]
- imm_i = {{20{instr[31]}}, instr[31:20]}
- imm_s = {{20{instr[31]}}, instr[31:25], instr[11:7]}
- imm_b = {{19{instr[31]}}, instr[31], instr[7], instr[30:25], instr[11:8], 1'b0}
- imm_u = {instr[31:12], 12'b0}
- imm_j = {{11{instr[31]}}, instr[31], instr[19:12], instr[20], instr[30:21], 1'b0}

商业建议

不要把所有立即数生成散落在 decode case 里，建议独立 immgen：

```
case (imm_type)
  IMM_I: imm = imm_i;
  IMM_S: imm = imm_s;
  IMM_B: imm = imm_b;
  IMM_U: imm = imm_u;
  IMM_J: imm = imm_j;
  default: imm = 32'b0;
endcase
```

控制结构体

- decode_ctrl_t
- valid
- illegal
- rs1_en
- rs2_en
- rd_en
- imm_type
- alu_op
- op1_sel
- op2_sel
- cmp_op
- branch
- jal
- jalr
- mem_req
- mem_we
- mem_size
- mem_unsigned
- wb_en
- wb_sel
- sys_op

R-type

特点:

- 读 rs1/rs2
- 写 rd
- 无立即数
- 无访存
- 结果来自 ALU

控制大致为:

○ Text

```
rs1_en  = 1
rs2_en  = 1
rd_en   = 1
op1_sel = RS1
op2_sel = RS2
wb_en   = 1
wb_sel  = WB_ALU
mem_req = 0
branch = 0
```

I-type

如 ADDI/ORI/XORI/...

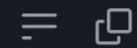
☐ Text



```
rs1_en    = 1
rs2_en    = 0
rd_en     = 1
imm_type  = IMM_I
op1_sel   = RS1
op2_sel   = IMM
wb_sel    = WB_ALU
```

Load

○ Text



```
rs1_en      = 1
rs2_en      = 0
rd_en       = 1
imm_type    = IMM_I
op1_sel     = RS1
op2_sel     = IMM
alu_op      = ADD      // 地址计算
mem_req     = 1
mem_we      = 0
mem_size    = ...
mem_unsigned = ...
wb_sel      = WB_MEM
wb_en       = 1
```

Store

○ Text



```
rs1_en    = 1
rs2_en    = 1
rd_en     = 0
imm_type  = IMM_S
op1_sel   = RS1
op2_sel   = IMM
alu_op    = ADD      // 地址计算
mem_req   = 1
mem_we    = 1
wb_en     = 0
```

Branch

比较可由：

- ALU复用
- 专用 comparator 完成。

商业设计中通常建议把branch compare从大 ALU 中逻辑拆开，减少控制复杂度。

○ Text

```
rs1_en    = 1
rs2_en    = 1
rd_en     = 0
imm_type  = IMM_B
branch    = 1
cmp_op    = ...
wb_en     = 0
mem_req   = 0
```

分支目标地址通常是：

○ Text

```
branch_target = pc + imm_b
```

JAL

○ Text

≡

□

```
rs1_en    = 0
rs2_en    = 0
rd_en     = (rd != 0) or simply 1
imm_type  = IMM_J
jal       = 1
wb_en     = 1
wb_sel    = WB_PC4
```

跳转目标:

○ Text

≡

□

```
pc + imm_j
```

↓

JALR

○ Text

rs1_en = 1

rs2_en = 0

rd_en = 1

imm_type = IMM_I

jalr = 1

wb_en = 1

wb_sel = WB_PC4

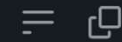
跳转目标:

○ Text

(rs1 + imm_i) & ~1

LUI

○ Text



```
rs1_en    = 0
rs2_en    = 0
rd_en     = 1
imm_type  = IMM_U
wb_en     = 1
wb_sel    = WB_ALU or WB_IMM
```

通常做法：

- ALU 输出 `imm_u`
- 或专门 bypass 立即数

AUIPC

```
○ Text

rs1_en    = 0
rs2_en    = 0
rd_en     = 1
imm_type  = IMM_U
op1_sel   = PC
op2_sel   = IMM
alu_op    = ADD
wb_sel    = WB_ALU
wb_en     = 1
```

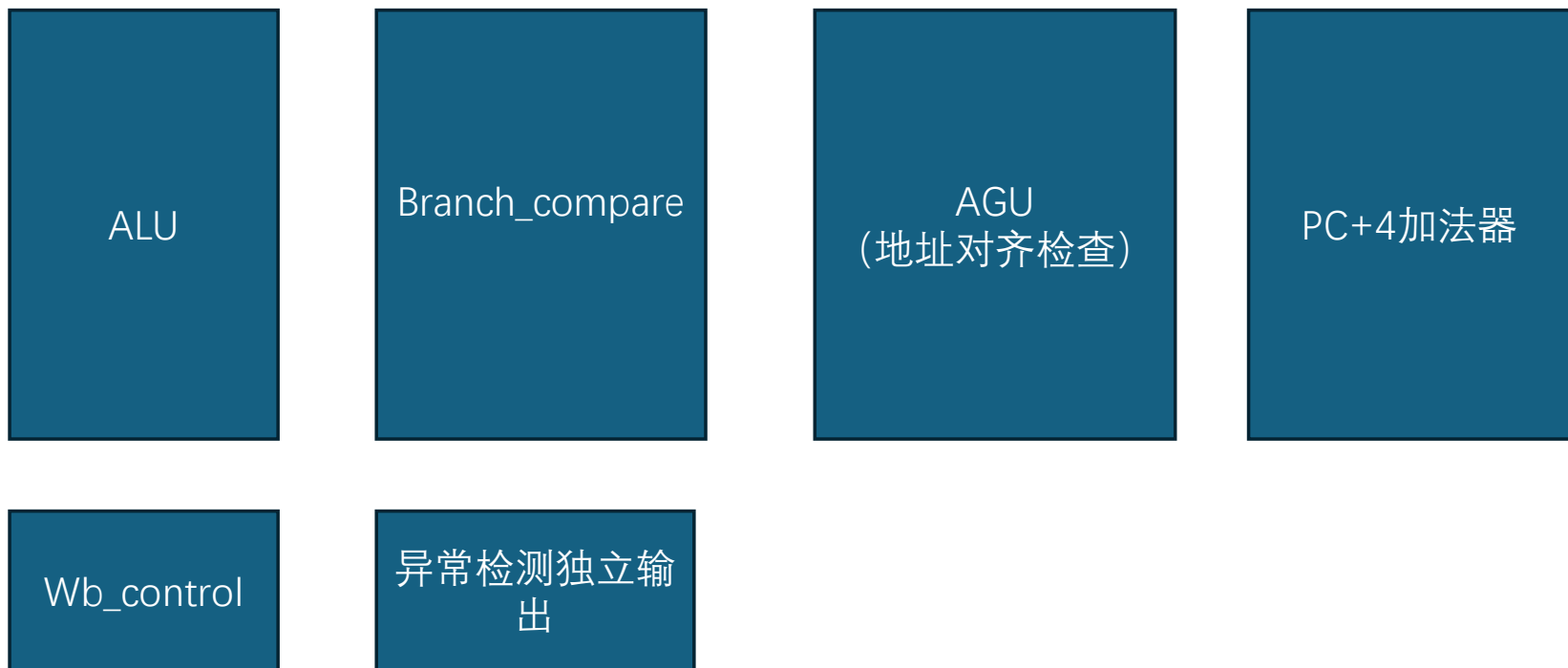
IDU_control

- ALU类: `DECODE -> EXEC -> WB`
- Branch/Jump: `DECODE -> EXEC -> FETCH`
- Load: `DECODE -> EXEC(addr) -> MEM -> WB -> FETCH`
- Store: `DECODE -> EXEC(addr) -> MEM -> FETCH`
- Illegal: `DECODE -> TRAP`

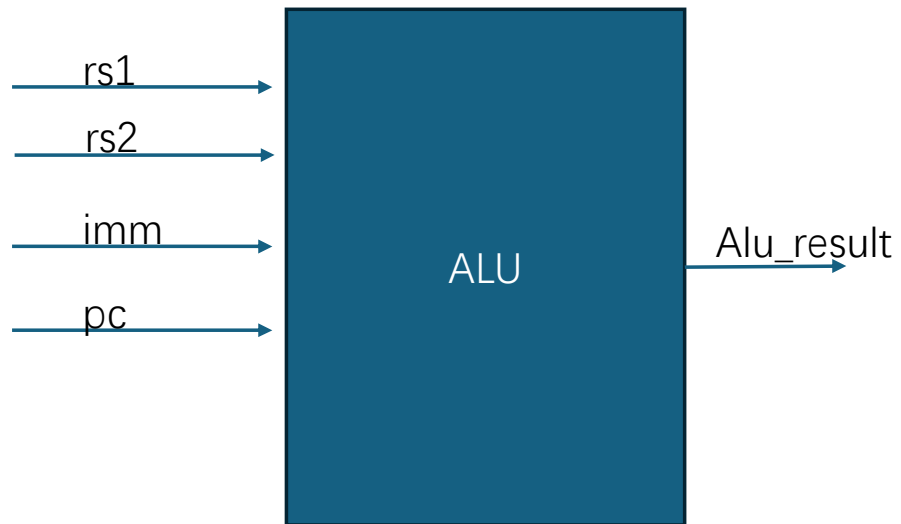
EXU

- 执行模块

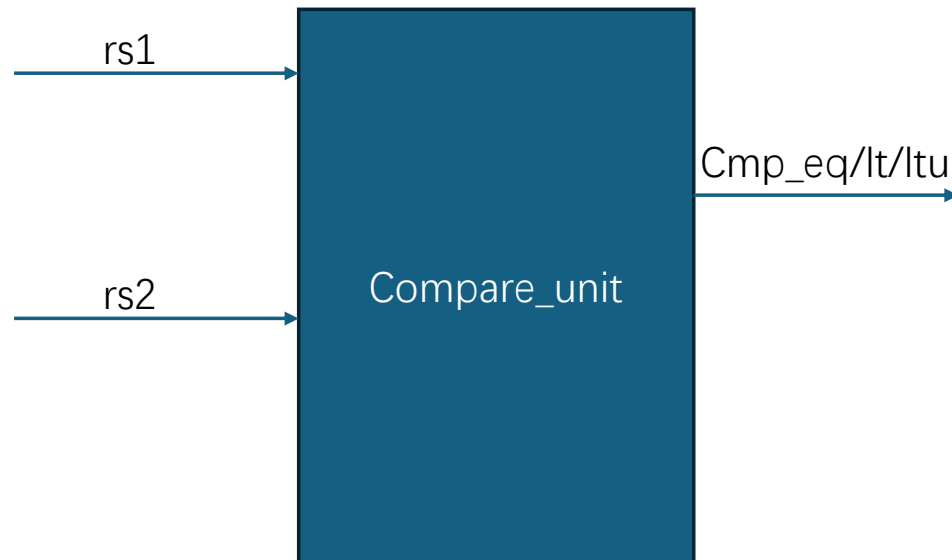
EXU_datapath



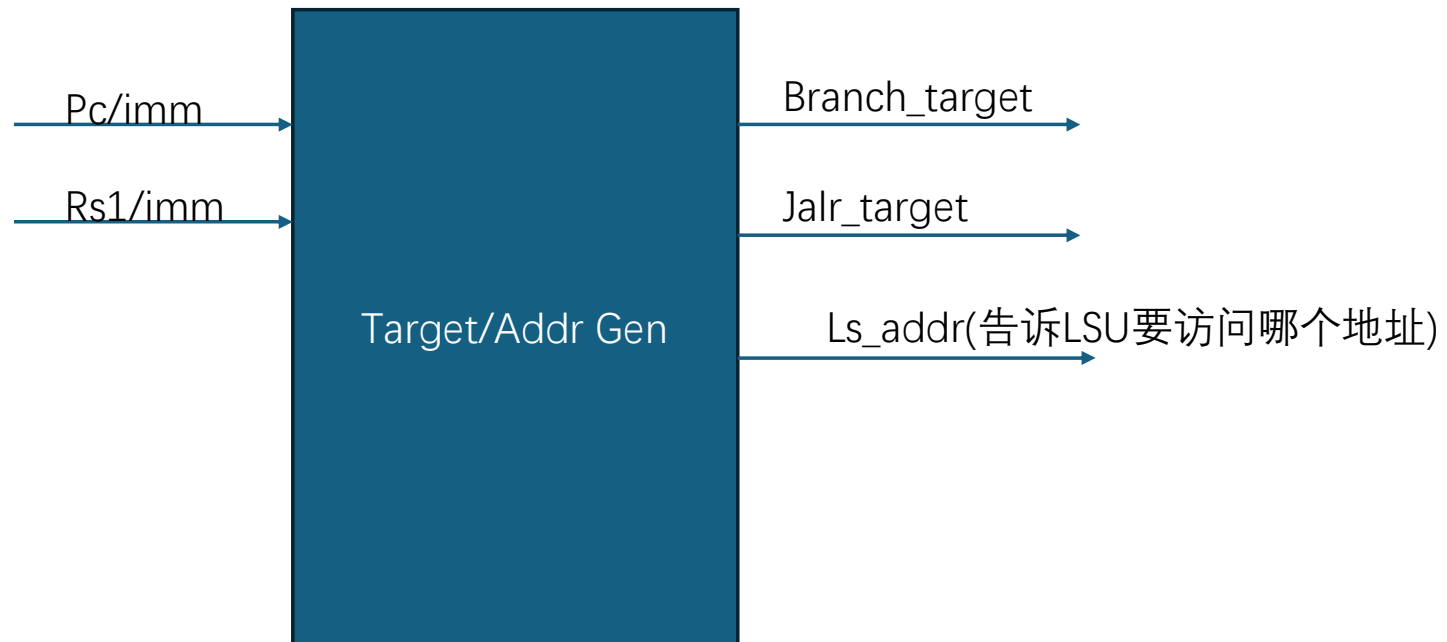
ALU



Compare unit

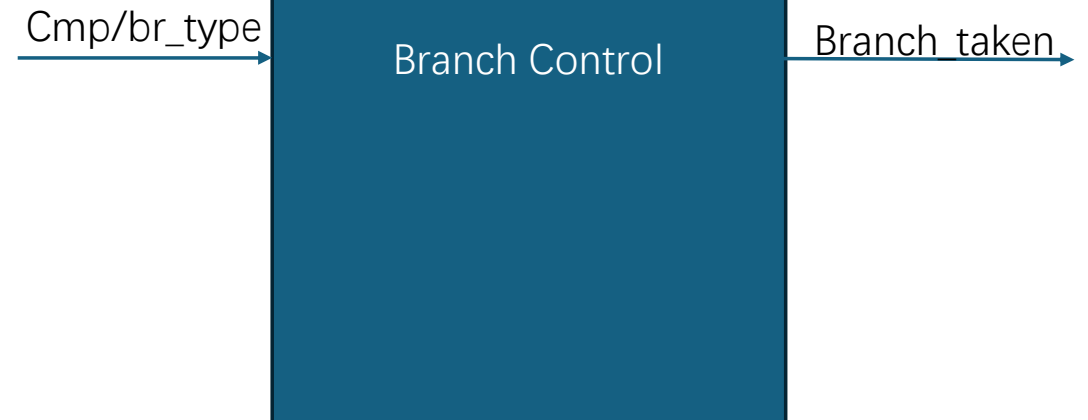


Target/Addr Gen(地址生成单元)

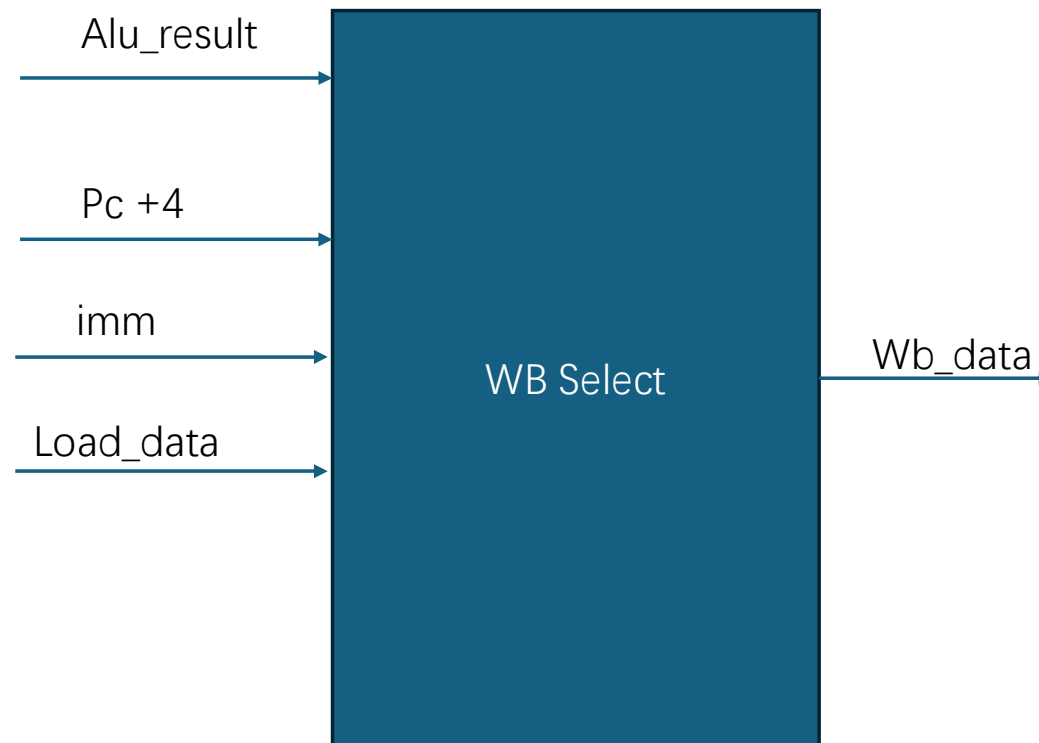


Branch Control

- 1.CMP:比较器输出结果
- 2.br_type: 分支类型控制信号
例如是不是BEQ\BNE\JAL之类



WB Select



In/out

输入

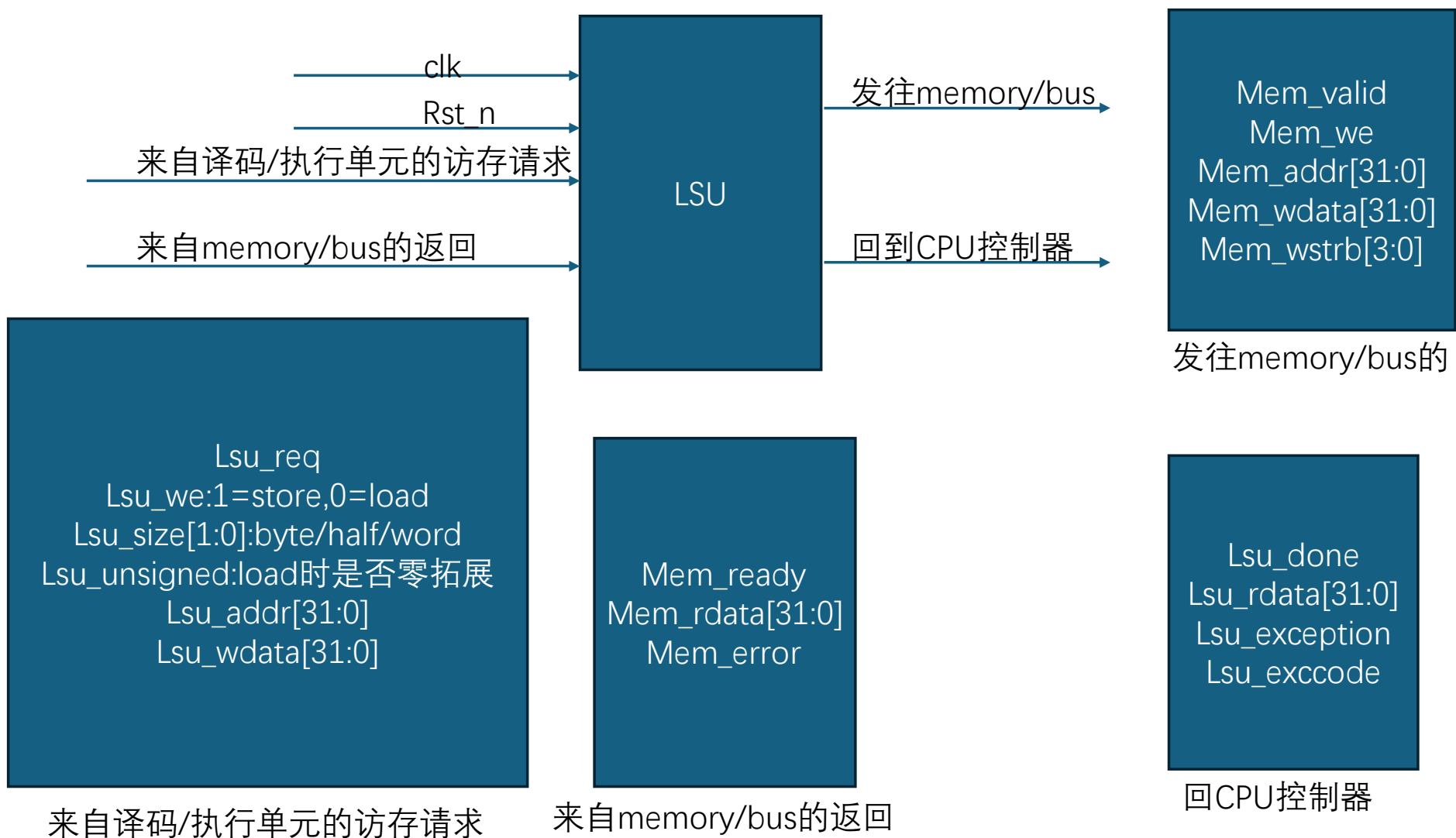
- `clk`, `rst_n`
- `ex_valid`
- `pc`
- `instr`
- `rs1_val`
- `rs2_val`
- `imm`
- `rd_idx`
- `alu_op`
- `cmp_op`
- `br_type`
- `jmp_type`
- `ls_type`
- `wb_sel`
- `signed/unsigned control`
- `csr/trap/debug control` (如果支持)

输出

- `ex_done`
- `alu_result`
- `branch_taken`
- `branch_target`
- `ls_addr`
- `store_data`
- `wb_data_candidate`
- `rd_we`
- `exception_valid`
- `exception_cause`
- `misalign`
- `jalr_target`
- 发给主控 FSM 的握手信号

LSU

- 负责根据控制信号控制存储器，从存储器中读出数据，或将数据写入寄存器，主要是负责访存，必须完整支持
LB/LH/LW/LBU/LHU/SB/SH/SW



6.2 Store 例子

SB

若 `addr[1:0] = 2'b10`，写一个字节：

- `mem_wstrb = 4'b0100`
- `mem_wdata = rs2[7:0] << 16`

SH

若 `addr[1:0] = 2'b10`：

- `mem_wstrb = 4'b1100`
- `mem_wdata = rs2[15:0] << 16`

SW

- `mem_wstrb = 4'b1111`
- `mem_wdata = rs2`

8.5 MMIO 访问的副作用

商业级CPU常把 LSU 同时接：

- data SRAM
- 外设寄存器区 (MMIO)

对 MMIO：

- 读可能有副作用
- 写可能需要等待外设 ready
- 不能因为控制器抖动重复发起请求

所以 `mem_valid` 与一次访存必须“一一对应”，防止重复写外设。

9.1 直接SRAM接口

适合小核：

- `cs`
- `we`
- `addr`
- `wdata`
- `be`
- `rdata`

如果 SRAM 固定单周期，LSU 很简单；但仍建议保留 `ready/valid` 抽象，便于以后升级。

11.1 控制与数据通路分离

- 控制：FSM、异常、握手
- 数据：地址、掩码、移位、扩展

这样可读性和验证性更好。

13.2 对齐异常

- LH/LHU/SH offset=1/3
- LW/SW offset=1/2/3

13.3 随机 wait state

memory ready 随机延迟，检查：

- 请求不丢
- 不重复发
- 返回后只完成一次

13.5 边界地址

例如：

- 0x00000000
- 0xFFFFFFFF
- 地址译码边界（SRAM/MMIO切换边界）

LSU侧

- 请求锁存
- 对齐检查
- 生成 addr/wdata/wstrb
- 发起统一 memory transaction
- 等待返回
- 生成 load result / exception
- 给控制器 done

Bus adapter侧

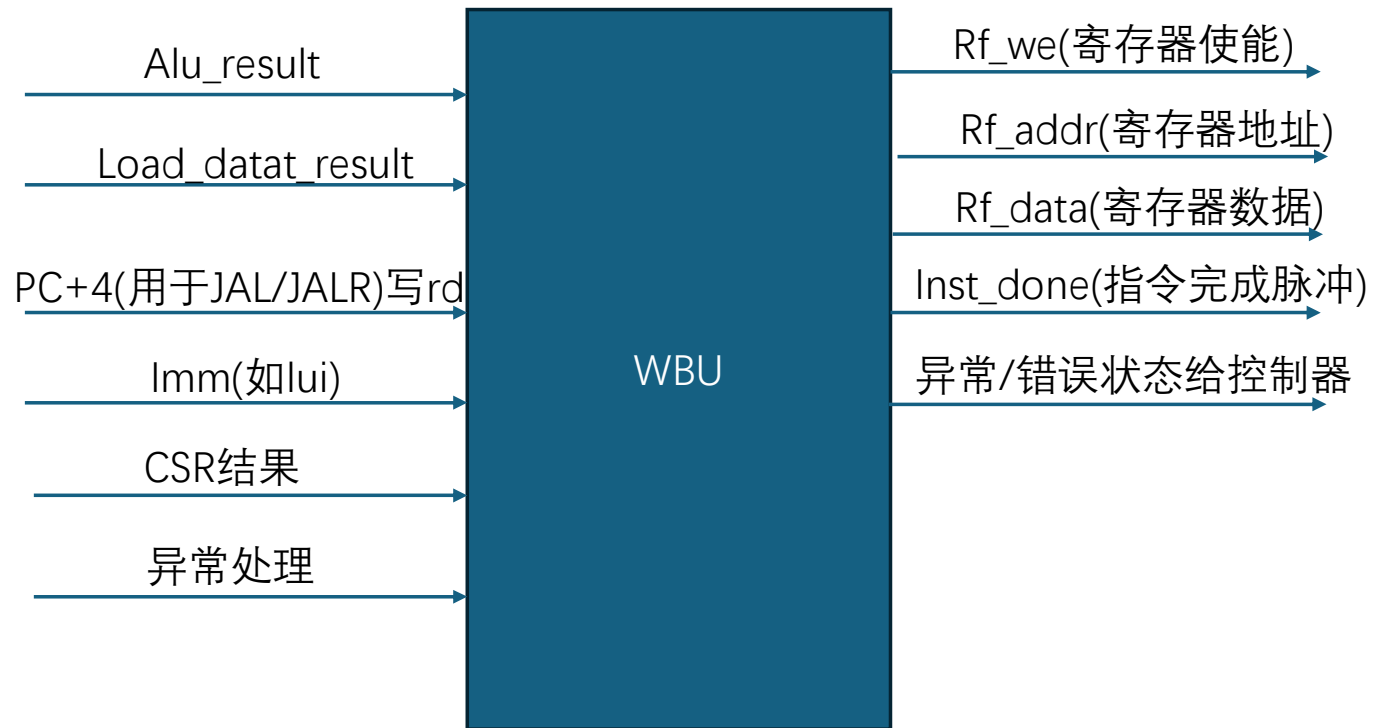
- 根据地址做 region decode
- 选择 SRAM / APB / AHB-Lite slave
- 处理具体协议
- 返回 ready/rdata/error

- 支持全部 RV32I 基本 load/store
- 严格地址对齐检查
- 明确异常上报
- 请求锁存
- 支持多周期存储器握手
- 字节使能与写数据 lane 对齐
- load 数据抽取与符号/零扩展
- 防止重复发起 MMIO 写
- 与具体总线协议解耦
- 便于验证与时序收敛

WBU:写回单元

- 在指令执行完成后，把最终结果可靠地写回寄存器堆，并处理好写回时序、写回来源选择、异常/无效写回屏蔽

datapath



control

- EXU先产生结果并锁存
- LSU若是load/store, 则等待握手完成
- 到WB状态时统一写回
- WBU不能直接原样写回, 必须按照指令做字节提取或者拓展
- 必须考虑以下导致不能写回的情况

模块架构

- 1.写回请求形成
- 2.提交仲裁/屏蔽
- 3.架构提交输出

wb: 注意事项

- 1.不建议在EXU/LSU直接写REG (1.异常屏蔽难做2.trace不集中3.后续加上CSR/debug很乱)
- 2.不建议在WBU同时做大量运算, 会使得WBU路径过长, 职责混乱
- 3.不区分“结果有效”和“运行提交”, result_valid和commit_enable逻辑上最好分离, 这在未来拓展中断的时候有用
- 4.WBU是提交点而不是计算点
- 5.所有执行结果先形成统一的wb_req
- 6.WBU统一完成寄存器写回、异常屏蔽、commit trace
- 7.最终写使能一定要屏蔽x0、异常、flush
- 8.给后续CSR/中断/debug/乘除法预留写回源编码